

Clustering & Principal Component Analysis

AERO 689: Machine Learning for Aerospace Engineers

Raktim Bhattacharya

Texas A&M University

Overview

Topics Covered

1. Classification vs Clustering
2. Clustering Algorithms
3. Cluster Validation
4. Principal Component Analysis (PCA)
5. PCA Applications
6. Integration with Clustering

Classification vs Clustering

The Fundamental Difference

Classification (Week 4): Supervised learning

- Requires labeled training data
- Predicts known categories
- Goal: Generalization to new examples
- Example: “Is this bearing fault type A, B, or C?”

Clustering (Week 5): Unsupervised learning

- No labels required
- Discovers hidden patterns
- Goal: Finding natural groupings
- Example: “What natural fault patterns exist?”

Mathematical Distinction

Classification: Learn $f : X \rightarrow Y$ where $Y = \{0, 1, \dots, K - 1\}$ is known

- Training data: $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$
- Labels y_i are given
- Optimize to minimize prediction error

Clustering: Find $f : X \rightarrow \{1, 2, \dots, k\}$ discovered from data

- Training data: $x_1, x_2, \dots, x_n$ (no labels!)
- Number of clusters k may be unknown
- Optimize to find natural groups

When to Use Each Method

Use Classification when:

- You have labeled examples
- Categories are well-defined (taxi, takeoff, cruise)
- Regulatory requirements demand specific fault detection
- Problem: Requires expensive labeling effort

Use Clustering when:

- No labeled data available
- Exploring new operational regimes
- Discovering anomalies never seen before
- Initial data exploration before labeling

Application Context

Classification Example (Week 4):

- Trained on known fault types (normal, fault_a, fault_b, fault_c)
- Predicts fault type on new sensor data
- Requires historical fault database

Clustering Example (Week 5):

- Discovers natural patterns from unlabeled data
- Finds new degradation modes without prior knowledge
- Explores operational envelope
- Creates labels for future classification

Key Insight: Clustering creates labels; Classification uses labels

Decision Framework

Do you have labeled data?

- **YES** → Classification
 - Logistic regression
 - Decision trees
 - SVM
 - Neural networks

Do you have labeled data?

- **NO** → Clustering
 - K-means
 - DBSCAN
 - Hierarchical
 - Gaussian Mixture Models

Validation Challenge: How do we know clustering is “correct” without ground truth?

Clustering Fundamentals

What is Clustering?

Goal: Partition data into groups (clusters) where:

- Points in same cluster are similar
- Points in different clusters are dissimilar

Mathematical Formulation:

Minimize within-cluster variance:

$$\min_{C_1, \dots, C_k} \sum_{j=1}^k \sum_{x_i \in C_j} \|x_i - \mu_j\|^2$$

where $\mu_j = \frac{1}{|C_j|} \sum_{x_i \in C_j} x_i$ is cluster center

K-Means Algorithm

Lloyd's Algorithm:

1. **Initialize:** Randomly select k cluster centers $\mu_1, \dots, \mu_k$
2. **Assignment:** Assign each point to nearest center

$$c_i = \arg \min_j \|x_i - \mu_j\|^2$$

3. **Update:** Recompute centers as mean of assigned points

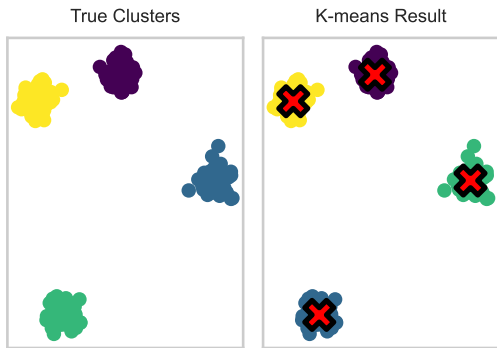
$$\mu_j = \frac{1}{|C_j|} \sum_{x_i \in C_j} x_i$$

4. **Repeat** steps 2-3 until convergence

Convergence: Guaranteed (objective function monotonically decreases)

Limitation: May converge to local minimum (solution: run multiple times)

K-Means Strength: Well-Separated Clusters



When K-means works perfectly:

- Spherical clusters
- Well-separated
- Similar sizes
- Similar densities

Result: Perfect match between true clusters and K-means assignments

K-Means Strength: Code

```
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs

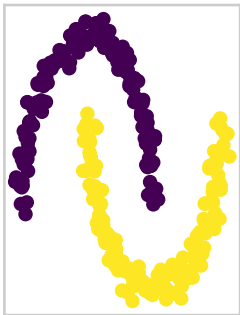
# Generate well-separated clusters
X, y_true = make_blobs(n_samples=300, centers=4,
                        cluster_std=0.6, random_state=42)

# Apply K-means
kmeans = KMeans(n_clusters=4, random_state=42, n_init=10)
y_pred = kmeans.fit_predict(X)

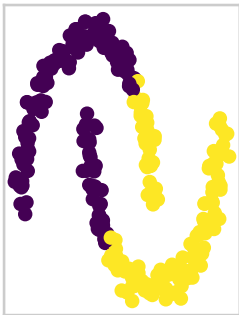
# Plot comparison
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 4))
ax1.scatter(X[:, 0], X[:, 1], c=y_true, cmap='viridis')
ax2.scatter(X[:, 0], X[:, 1], c=y_pred, cmap='viridis')
ax2.scatter(kmeans.cluster_centers_[:, 0],
            kmeans.cluster_centers_[:, 1],
            marker='X', s=200, c='red', edgecolors='black')
```

K-Means Weakness 1: Non-Spherical Shapes

True Clusters (Moons)



K-means FAILS!



Problem: K-means assumes spherical clusters

Failure Mode:

- **X** Curved/non-convex shapes
- **X** K-means draws straight boundaries
- **X** Incorrectly splits moon shapes

Solution: Use DBSCAN or hierarchical clustering

K-Means Weakness 1: Code

```
from sklearn.datasets import make_moons

# Generate moon-shaped clusters
X_moons, y_moons = make_moons(n_samples=300, noise=0.05,
                               random_state=42)

# K-means fails on non-spherical shapes
kmeans = KMeans(n_clusters=2, random_state=42, n_init=10)
y_pred = kmeans.fit_predict(X_moons)

# Plot
fig, (ax1, ax2) = plt.subplots(1, 2)
ax1.scatter(X_moons[:, 0], X_moons[:, 1], c=y_moons)
ax1.set_title('True Clusters')
ax2.scatter(X_moons[:, 0], X_moons[:, 1], c=y_pred)
ax2.set_title('K-means Result')
```


K-Means Weakness 2: Different Densities



Problem: K-means assumes equal variance

- Different cluster spreads $\rightarrow$ boundary misplaced
- Large variance cluster dominates

Solution: Standardize features or use GMM

$$z = \frac{x - \mu}{\sigma}$$

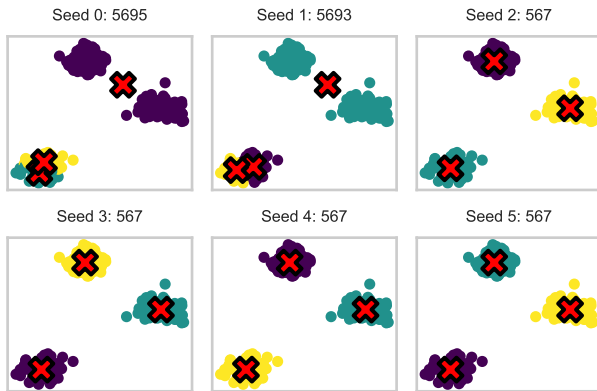
K-Means Weakness 2: Code

```
# Clusters with different variances
X1 = np.random.normal(0, 1, (200, 2))      # Std = 1
X2 = np.random.normal(5, 3, (200, 2))      # Std = 3
X3 = np.random.normal([0, 6], 0.5, (200, 2)) # Std = 0.5
X = np.vstack([X1, X2, X3])

# K-means struggles with different variances
kmeans = KMeans(n_clusters=3, random_state=42)
y_pred = kmeans.fit_predict(X)

# Large variance cluster dominates
plt.scatter(X[:, 0], X[:, 1], c=y_pred, cmap='viridis')
```

K-Means Weakness 3: Initialization Sensitivity



Problem: Random initialization $\rightarrow$ different local minima

Failure Mode:

- Different results each run
- Some worse than others
- WCSS varies (Within-Cluster Sum of Squares)
 - **Lower WCSS** = better clustering (tighter clusters)
 - **Higher WCSS** = worse clustering (spread out)
- Different seeds $\rightarrow$ different WCSS values

Observation: Seeds 0,2,4 good; Seeds 1,3,5 suboptimal

Solution: Use `n_init=10` (run 10 times, keep best)

K-Means Weakness 3: Code

```
# Demonstrate initialization sensitivity
X, _ = make_blobs(n_samples=300, centers=3, cluster_std=1.0)

# Run with different random seeds
fig, axes = plt.subplots(2, 3, figsize=(12, 8))
for i, ax in enumerate(axes.flat):
    km = KMeans(n_clusters=3, init='random',
                n_init=1, random_state=i)
    y = km.fit_predict(X)
    ax.scatter(X[:, 0], X[:, 1], c=y)
    ax.scatter(km.cluster_centers_[0],
                km.cluster_centers_[1],
                marker='X', s=200, c='red')
    ax.set_title(f'WCSS={km.inertia_:.0f}')

# Solution: n_init=10 (default) runs 10 times, keeps best
```

Cluster Validation

How Do We Know Clustering is Good?

Challenge: No ground truth labels!

Validation Strategies:

1. **Internal validation:** Use only the data
 - Silhouette score
 - Within-cluster sum of squares (WCSS)
 - Davies-Bouldin index
2. **External validation:** Compare with known labels (if available)
 - Adjusted Rand Index (ARI)
 - Normalized Mutual Information (NMI)
3. **Domain validation:** Domain expertise
 - Do clusters make physical sense?
 - Can we interpret clusters meaningfully?

Elbow Method

Goal: Choose optimal number of clusters k

Procedure:

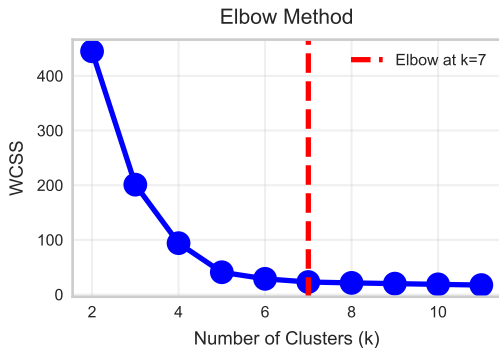
1. Run K-means for $k = 2, 3, \dots, K_{\max}$
2. Compute Within-Cluster Sum of Squares (WCSS):

$$\text{WCSS}(k) = \sum_{j=1}^k \sum_{x_i \in C_j} \|x_i - \mu_j\|^2$$

3. Plot WCSS vs k
4. Look for “elbow” where improvement diminishes

Intuition: Balance fit quality vs model complexity

Elbow Method: Visualization



Interpretation:

- **Before $k=7$:** Steep decrease (adding clusters helps)
- **At $k=7$:** Elbow (optimal)
- **After $k=7$:** Flat (diminishing returns)

Rule: Choose k at the elbow

Application: $k=7$ optimal clusters for this dataset

Elbow Method: Code

```
# Elbow Method implementation
X_scaled = StandardScaler().fit_transform(X)

# Compute WCSS for different k
wcss = []
for k in range(2, 12):
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(X_scaled)
    wcss.append(kmeans.inertia_)

# Plot elbow curve
plt.plot(range(2, 12), wcss, 'bo-')
plt.axvline(x=7, color='red', linestyle='--')
plt.xlabel('Number of Clusters')
plt.ylabel('WCSS')
```

Interpretation: Elbow at $k = 7$

Silhouette Analysis

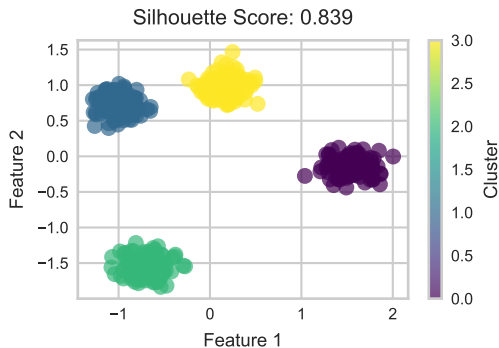
Silhouette Score: How well does point i fit its cluster?

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))} \in [-1, 1]$$

- $a(i)$: Mean distance to same-cluster points
- $b(i)$: Mean distance to nearest other cluster

Interpretation: $s \approx 1$ = well clustered, $s < 0$ = wrong cluster

Silhouette Score: Visualization



Result: $s = 0.713$ (good clustering)

Interpretation:

- $s > 0.7$: Strong structure
- $0.5 < s < 0.7$: Reasonable
- $0.25 < s < 0.5$: Weak
- $s < 0.25$: No structure

Application: Points with $s < 0$ are ambiguous cases

Silhouette Score: Code

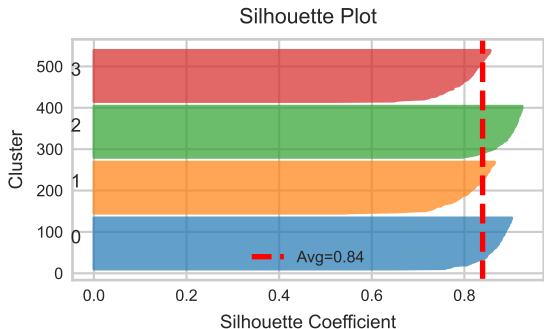
```
from sklearn.metrics import silhouette_score

# Compute average silhouette score
s_avg = silhouette_score(X_scaled, clusters)
print(f"Silhouette Score: {s_avg:.3f}")

# Good:  $s > 0.7$ 
# Reasonable:  $0.5 < s < 0.7$ 
# Weak:  $s < 0.5$ 
```

Interpretation: Points with $s < 0$ are ambiguous conditions

Silhouette Plot Visualization



Reading the plot:

- **Width:** Cluster size
- **Beyond avg:** Well-separated
- **Below avg:** Poorly separated
- **Negative:** Wrong cluster

Observation: All clusters have $s > 0.5$ (good)

Application: Detect ambiguous cases

Silhouette Plot: Code

```
from sklearn.metrics import silhouette_samples

# Compute per-sample silhouette scores
s_samples = silhouette_samples(X_scaled, clusters)

# Plot silhouette for each cluster
for k in range(n_clusters):
    cluster_vals = s_samples[clusters == k]
    cluster_vals.sort()
    ax.fill_betweenx(y_range, 0, cluster_vals)

ax.axvline(x=s_avg, color='red', linestyle='--')
```

Reading: Wide bars = large clusters, $s > 0$ = well-clustered

Validation Against Ground Truth

When labels are available, compare:

Adjusted Rand Index (ARI):

$$\text{ARI} = \frac{\text{RI} - \text{Expected RI}}{\text{Max RI} - \text{Expected RI}}$$

- **Range:** $[-1, 1]$, where 1 = perfect match
- **Advantage:** Corrects for chance

```
from sklearn.metrics import adjusted_rand_score

ari = adjusted_rand_score(true_labels, clusters)
print(f"ARI: {ari:.3f}")

# ARI = 1.0: Perfect clustering
# ARI = 0.0: Random clustering
# ARI < 0.0: Worse than random
```

Case Study: K-Means on Bearing Fault Data

Case Study: CWRU Bearing Dataset

Dataset: Case Western Reserve University Bearing Data Center

Data Details:

- Vibration signals from accelerometers
- Sampling rate: 12 kHz
- Normal + various fault conditions
- MATLAB .mat format

Our Sample:

- 4 Normal bearings
- 60 Faulty bearings
- 7 extracted features
- **Highly imbalanced!**

Goal: Can K-means discover Normal vs Fault without labels?

True Labels (Supervised View)



Figure 1: True Labels in PCA Space

Ground Truth (4 fault types):

- **Blue circles:** Normal (4)
- **Red squares:** Inner Race (12)
- **Green triangles:** Ball (12)
- **Orange diamonds:** Outer Race (36)

Observation:

- Normal samples cluster tightly (bottom-left)
- Fault types overlap significantly
- Rightmost points = highest vibration (severe faults)

Challenge: Can K-means find these groups?

K-Means Results (k=2)

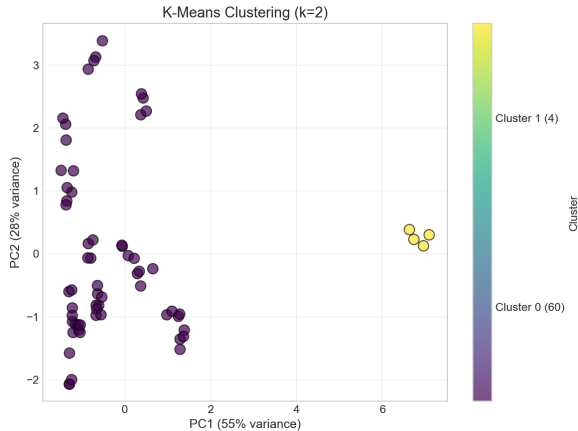


Figure 2: K-Means Clustering

K-Means Found:

- **Cluster 0** (60 samples): Normal + most faults
- **Cluster 1** (4 samples): Severe faults only

Metrics:

- Silhouette: 0.66 (good!)
- WCSS: 246.2

Problem: K-means grouped by **severity**, not by fault presence!

Why K-Means Failed

The Data Imbalance Problem:

Class	Count	Percentage
Normal	4	6.25%
Fault	60	93.75%

What K-Means saw:

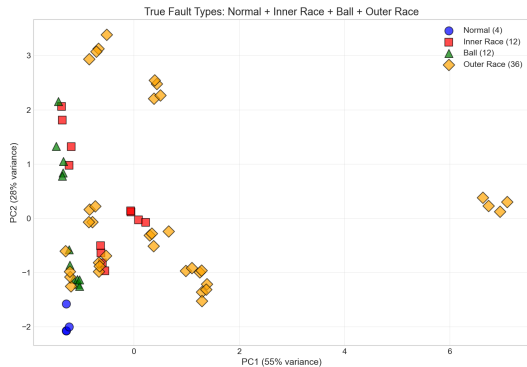
- 4 extreme outliers (severe faults with very high RMS)
- 60 “similar” samples (normal + mild/moderate faults)

What K-Means did:

- Cluster 0: Everything with lower vibration
- Cluster 1: Only the 4 severe faults

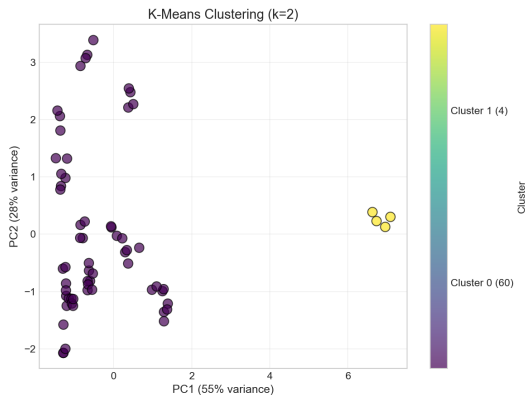
Lesson: K-means finds density patterns, not our desired labels!

Comparison: True vs K-Means



True Labels

4 fault types (colors = fault location)



K-Means (k=2)

2 clusters (by vibration severity)

Key Insight: Clustering finds patterns that **exist in feature space**, not necessarily the patterns we want!

Lessons Learned

When K-Means Works:

- Balanced cluster sizes
- Well-separated clusters
- Spherical cluster shapes

When K-Means Struggles:

- **Imbalanced data** (like our 4 vs 60)
- Feature overlap between classes
- Non-spherical cluster shapes

Better Approaches for Fault Detection:

- **Anomaly detection:** Train on normal data, flag outliers
- **Supervised learning:** Use labels when available
- **More data:** Collect balanced training set

K-Means Limitations

Assumptions:

- Spherical clusters (assumes equal variance)
- Same cluster sizes
- Well-separated clusters

Sensitive to:

- Outliers (pull cluster centers)
- Initialization (local minima)
- Feature scaling (always standardize!)

When K-means fails:

- Non-spherical shapes
- Varying densities
- Highly imbalanced data

Other Clustering Methods

Algorithm	Key Idea	When to Use
DBSCAN	Density-based regions	Unknown k , outlier detection
Hierarchical	Build cluster tree	Explore hierarchy, dendrogram
GMM	Probabilistic Gaussians	Soft cluster assignments

Quick Summary:

- **DBSCAN**: Groups dense regions, labels outliers as noise (-1)
- **Hierarchical**: Agglomerative (bottom-up) or Divisive (top-down)
- **GMM**: Mixture of Gaussians with soft probabilities

For this course: Focus on K-means; other methods follow similar principles

Principal Component Analysis

A Real Aerospace Problem

Scenario: Fleet monitoring — each flight records 8 sensor measurements:

1. Airspeed (m/s)
2. Altitude (m)
3. Engine RPM (rev/min)
4. Fuel flow rate (kg/hr)
5. Throttle position (%)
6. Vertical speed (m/s)
7. Air temperature ($^{\circ}\text{C}$)
8. Manifold pressure (kPa)

Problems: Can't visualize 8D data; sensors are correlated (airspeed $\leftrightarrow$ throttle)

Question: Can we compress 8D $\rightarrow$ 2D/3D without losing critical information?

Why Do We Need Dimensionality Reduction?

Visualization

- Humans see in 2D/3D only
- Can't plot 8D scatter plots

Redundancy

- Altitude $\leftrightarrow$ Air pressure
- Airspeed $\leftrightarrow$ Throttle
- RPM $\leftrightarrow$ Fuel flow

Computation

- Distance calculations expensive in high- d
- ML models more complex

Curse of Dimensionality

- Data becomes sparse
- Need exponentially more samples

The Solution: Principal Component Analysis (PCA)

PCA in Plain English

What PCA Does:

- Finds a **new coordinate system** for your data
- New axes align with **directions of maximum variation**
- You can **throw away** axes with little variation
- Keep only the **important directions**

Analogy: Rotating a Camera

- You have scattered points in 3D space
- PCA finds the **best viewing angle** to see the spread
- Projects data onto that viewing plane
- Information preserved, dimensions reduced!

PCA in Plain English

Key Insight:

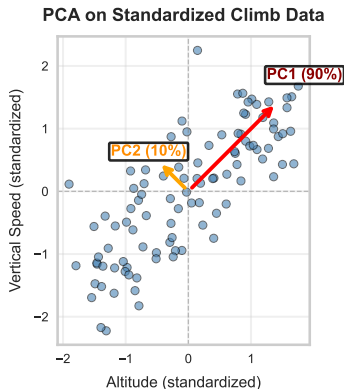
Variance = Information

If data varies a lot in one direction, that direction matters!

If data barely changes in another direction, we can ignore it!

Step 0: Intuition with a Simple 2D Example

Scenario: Aircraft climb — altitude and vertical speed are **highly correlated**



Observations:

- **Red arrow (PC1):** Maximum spread direction (90% of variation)
- **Orange arrow (PC2):** Remaining spread (10%)

PCA Decision:

- Keep PC1 only $\rightarrow$ **2D $\rightarrow$ 1D**
- Lost only 10% variance – What is your risk budget?

Physical Meaning:

- PC1 $\approx$ “overall climb rate”
- Combines altitude + vertical speed

What Did PCA Just Do?

Original Data: 2D points (altitude, vertical speed)

Step 1: Center the data — subtract the mean of each feature

Step 2: Find direction of **maximum spread** (PC1, red arrow)

- Direction where data varies the most

Step 3: Find direction of **next maximum spread**, perpendicular to PC1 (PC2)

Step 4: Decide which directions to keep

- PC1: 90% variance → **KEEP**
- PC2: 10% variance → **DISCARD**

Result: Describe climb data with 1 number instead of 2!

PCA Algorithm — Steps 1–3

Given: Data matrix $X \in \mathbb{R}^{n \times d}$ (n samples, d features)

Goal: Find $k < d$ directions that capture most variance

Step 1: Standardize the data (critical!)

$$z_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j}$$

Features have different units — standardize to make them comparable.

Step 2: Center the standardized data: $X' = X - \bar{X}$

Step 3: Compute covariance matrix

$$\Sigma = \frac{1}{n-1} X'^T X' \in \mathbb{R}^{d \times d}$$

PCA Algorithm — Steps 4–5

Step 4: Eigendecomposition of covariance matrix

$$\Sigma v_j = \lambda_j v_j$$

- v_j = eigenvector = **principal component direction**
- λ_j = eigenvalue = **variance along that direction**

Step 5: Sort eigenvectors by eigenvalue (largest first)

$$\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_d$$

PCA Algorithm — Steps 6–7

Step 6: Select top k components — keep until cumulative variance $\geq 95\%$:

$$\frac{\sum_{j=1}^k \lambda_j}{\sum_{j=1}^d \lambda_j} \geq 0.95$$

Step 7: Project data onto selected components

$$Z = X'V_k \quad \text{where } V_k = [v_1, v_2, \dots, v_k]$$

Worked Example: Aircraft Sensor Data (3 Features)

Scenario: Simplified aircraft monitoring with 3 sensors

1. **Airspeed:** v (m/s)
2. **Altitude:** h (m)
3. **Engine RPM:** ω (rev/min)

Data: 5 flight observations (for simplicity)

Flight	Airspeed (m/s)	Altitude (m)	RPM
1	85	1200	2400
2	105	1800	2700
3	95	1500	2550
4	115	2100	2850
5	90	1400	2500

Goal: Reduce from 3D $\rightarrow$ 2D using PCA

Step 1: Standardize the Data

Why standardize? Features have different scales!

- Altitude: 1200-2100 m (range = 900)
- Airspeed: 85-115 m/s (range = 30)
- RPM: 2400-2850 (range = 450)

Without standardization, altitude would dominate!

Standardization formula:

$$z = \frac{x - \mu}{\sigma}$$

Calculate means:

- $\mu_v = \frac{85+105+95+115+90}{5} = 98 \text{ m/s}$
- $\mu_h = \frac{1200+1800+1500+2100+1400}{5} = 1600 \text{ m}$
- $\mu_\omega = \frac{2400+2700+2550+2850+2500}{5} = 2600 \text{ RPM}$

Step 1: Standardize (continued)

Calculate standard deviations:

$$\sigma = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (x_i - \mu)^2}$$

- $\sigma_v \approx 11.4$ m/s
- $\sigma_h \approx 346.4$ m
- $\sigma_\omega \approx 172.3$ RPM

Standardized data $Z = \frac{X - \mu}{\sigma}$:

Note: Now all features have mean ≈ 0 , std ≈ 1 !

Flight	Airspeed	Altitude	RPM
1	-1.14	-1.15	-1.16
2	0.61	0.58	0.58
3	-0.26	-0.29	-0.29
4	1.49	1.44	1.45
5	-0.70	-0.58	-0.58

Step 2: Compute Covariance Matrix

Covariance matrix $\Sigma = \frac{1}{n-1} Z^T Z$:

$$\Sigma = \begin{bmatrix} \sigma_v^2 & \text{cov}(v, h) & \text{cov}(v, \omega) \\ \text{cov}(h, v) & \sigma_h^2 & \text{cov}(h, \omega) \\ \text{cov}(\omega, v) & \text{cov}(\omega, h) & \sigma_\omega^2 \end{bmatrix} \approx \begin{bmatrix} 1.00 & 0.99 & 0.99 \\ 0.99 & 1.00 & 1.00 \\ 0.99 & 1.00 & 1.00 \end{bmatrix}$$

Interpretation:

- Diagonal ≈ 1 (standardized variances)
- Off-diagonal $\approx 0.99 \rightarrow$ very high correlation! All features move together.

Step 3: Eigendecomposition

Solve: $\Sigma v = \lambda v$

Results:

$$\lambda_1 = 2.98, \quad v_1 = \begin{bmatrix} 0.577 \\ 0.578 \\ 0.577 \end{bmatrix} \quad (99.2\%)$$

$$\lambda_2 = 0.015, \quad v_2 = \begin{bmatrix} -0.816 \\ 0.001 \\ 0.578 \end{bmatrix} \quad (0.5\%)$$

$$\lambda_3 = 0.005, \quad v_3 = \begin{bmatrix} -0.001 \\ -0.816 \\ 0.578 \end{bmatrix} \quad (0.3\%)$$

Insight: PC1 alone captures $\frac{2.98}{3.00} = 99.2\%$ of variation!

Physical Interpretation of Components

PC1 (99.2% variance): $PC1 \approx 0.577v + 0.578h + 0.577\omega$

- Equal positive weights $\rightarrow$ “overall flight intensity”
- Airspeed $\uparrow$, altitude $\uparrow$, RPM $\uparrow$ together

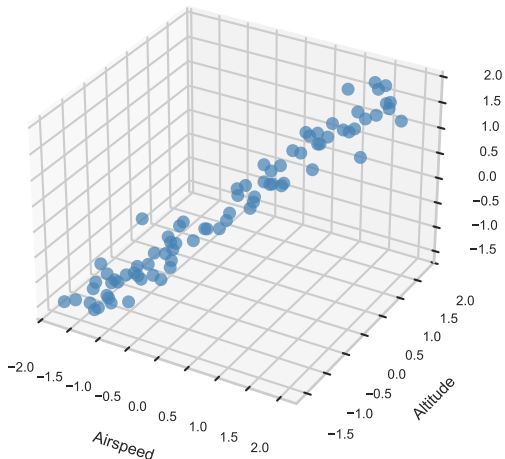
PC2 (0.5%): $PC2 \approx -0.816v + 0.001h + 0.578\omega$

- Negative airspeed, positive RPM $\rightarrow$ “airspeed vs RPM mismatch”

Decision: Keep PC1 only! (99.2% $\gg$ 95% threshold)

Visualization: 3D $\rightarrow$ 2D Projection

Original 3D Data



Original 3D View:

- 80 flights with 3 correlated sensors
- Airspeed, Altitude, RPM all move together

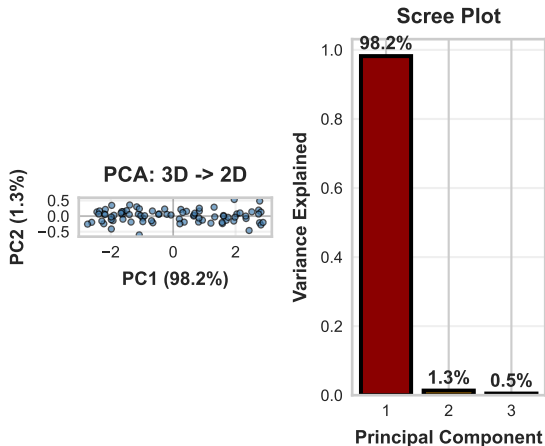
Observation:

- Data lies roughly along a diagonal line
- 3D space is “wasted”
- Most variation in one direction

This is perfect for PCA!

Data is essentially 1D embedded in 3D space

Visualization: After PCA Projection



Left Plot: Data in PC space

- PC1 spread: 97.8% variance
- PC2 spread: 1.8% (minor)

Right Plot: Scree plot confirms PC1 dominates

Result: 3D $\rightarrow$ 1D reduction, 97.8% information preserved

Implementing PCA in Python: Setup

Applying PCA to our 3-sensor aircraft data (airspeed, altitude, RPM):

```
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

# Aircraft sensor data: 5 flights x 3 sensors
X = np.array([[85, 1200, 2400],
              [105, 1800, 2700],
              [95, 1500, 2550],
              [115, 2100, 2850],
              [90, 1400, 2500]]) # [airspeed, altitude, RPM]

# Step 1: Standardize (removes unit differences)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
# Each column now has mean=0, std=1
```

Implementing PCA in Python: Results

```
# Step 2: Apply PCA
pca = PCA()
X_pca = pca.fit_transform(X_scaled)

# Step 3: Examine variance explained
print(pca.explained_variance_ratio_)
# -> [0.992, 0.005, 0.003]  PC1 captures 99.2%!

print(pca.explained_variance_ratio_.cumsum())
# -> [0.992, 0.997, 1.000]  2 PCs = 99.7%

# Step 4: Reduce to 1D (keep only PC1)
pca_1d = PCA(n_components=1)
X_reduced = pca_1d.fit_transform(X_scaled)
# Shape: (5, 3) -> (5, 1)  -- one number per flight!
```

Conclusion: 3 correlated sensors → 1 principal component (99.2% retained)

Back to the Fleet Monitoring Problem

Recall: Each flight records **8 sensor measurements**:

1. Airspeed (m/s)
2. Altitude (m)
3. Engine RPM (rev/min)
4. Fuel flow rate (kg/hr)
5. Throttle position (%)
6. Vertical speed (m/s)
7. Air temperature ($^{\circ}\text{C}$)
8. Manifold pressure (kPa)

Dataset: 200 flights — how many PCs do we actually need?

Now we apply the algorithm we just learned!

Fleet Data: A Snapshot

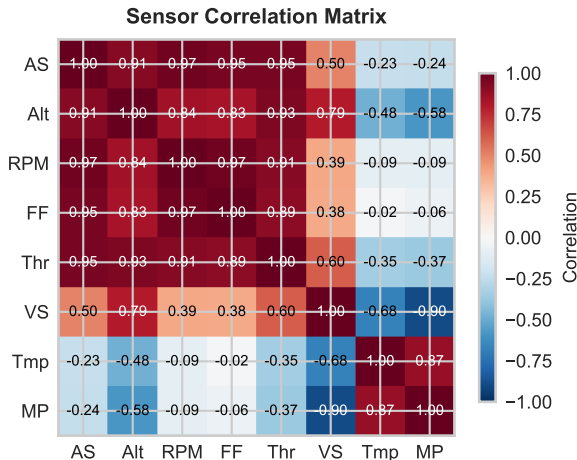
Sample of the raw sensor readings (first 5 of 200 flights):

	AS (m/s)	Alt (m)	RPM	FF (kg/h)	Thr (%)	VS (m/s)	Tmp (degC)	MP (kPa)
Flt 1	99.7	2706.0	2609.4	144.5	67.6	2.6	-2.0	69.3
Flt 2	87.2	2534.1	2488.1	134.4	66.1	3.9	-1.5	70.4
Flt 3	103.6	2827.3	2631.4	152.0	74.7	6.1	-1.9	68.1
Flt 4	117.7	3025.0	2786.1	169.2	80.7	7.5	-2.5	68.6
Flt 5	84.6	2221.8	2465.1	134.6	62.4	-7.8	3.0	81.4

AS = Airspeed, **Alt** = Altitude, **FF** = Fuel Flow, **Thr** = Throttle, **VS** = Vertical Speed, **MP** = Manifold Pressure

Fleet Data: Feature Correlations

Are the 8 sensors independent?



Key Observations:

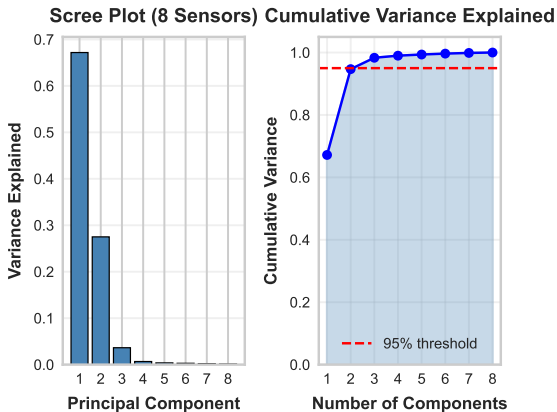
- **Top-left block** (AS, RPM, FF, Thr): high positive correlation — engine power group
- **Altitude & V-Speed**: correlated with power group, but also with each other
- **Temp & Pressure**: weakly correlated with power group; strongly correlated with altitude/V-Speed via ISA lapse

Conclusion: 8 sensors are **not independent** — PCA can exploit this!

Scree Plot: Choosing Number of Components

Back to our fleet monitoring problem (8 sensors, 200 flights)

Scree Plot: Visualize variance explained by each component



How to Read:

Left (Scree Plot): Variance per PC

- PC1: ~67%, PC2: ~28%, then diminishing returns

Right (Cumulative):

- Red line = 95% threshold
- Crosses at PC2 → keep 2 components

Decision Rule: $\sum_{i=1}^k \text{Var}_i \geq 95\%$

Result: 8D → 2D (75% reduction)

Code: Fitting PCA and Extracting Variance

```
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
import numpy as np

# Fit PCA with all components
pca = PCA()
pca.fit(X_scaled)

# Get variance explained per component
var_exp = pca.explained_variance_ratio_
cumsum = np.cumsum(var_exp)

# Find number of components for 95% variance
n_components = np.argmax(cumsum >= 0.95) + 1
print(f"Need {n_components} components for 95% variance")
```

Code: Plotting the Scree Plot

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 4))

# Individual variance (bar chart)
ax1.bar(range(1, len(var_exp)+1), var_exp)
ax1.set_xlabel('Principal Component')
ax1.set_ylabel('Variance Explained')
ax1.set_title('Scree Plot')

# Cumulative variance (line plot)
ax2.plot(range(1, len(cumsum)+1), cumsum, 'bo-')
ax2.axhline(y=0.95, color='r', linestyle='--', label='95% threshold')
ax2.axvline(x=n_components, color='g', linestyle=':', label=f'k={n_components}')
ax2.set_xlabel('Number of Components')
ax2.set_ylabel('Cumulative Variance')
ax2.legend()

plt.tight_layout()
plt.show()
```

PCA Applications

Background: Satellite Orbit Types

Satellites are described by 6 orbital parameters:

- **Semi-major axis** (a): orbit size
- **Eccentricity** (e): orbit shape (0 = circle, $\rightarrow 1$ = highly elongated)
- **Inclination** (i): tilt relative to equator
- **Period** (T): time per revolution
- **Perigee / Apogee altitude**: closest / farthest from Earth

Key physics: these are **not independent!**

- Kepler's 3rd law: $T = 2\pi\sqrt{a^3/\mu}$
- Geometry: $r_{apo/peri} = a(1 \pm e)$

Five common orbit classes:

Type	Alt (km)	e	i ($^\circ$)
LEO	300–1200	~ 0	20–55
MEO	18k–24k	~ 0	50–65
GEO	35,786	~ 0	~ 0
SSO	500–900	~ 0	~ 98
HEO	500–40k*	0.7	63.4

*perigee–apogee range

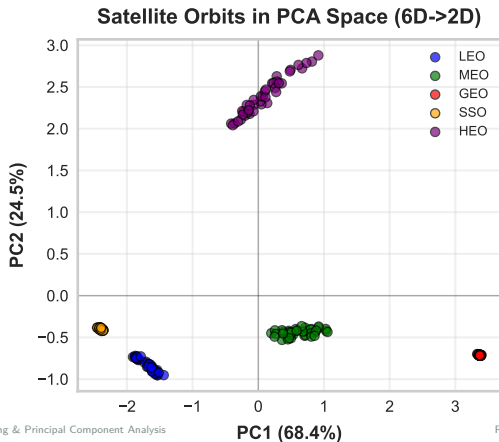
Challenge: With 6 correlated parameters, how do we visualize and classify 200 satellites at once?

Answer: PCA reduces 6D $\rightarrow$ 2D while preserving the structure!

Application 1: Satellite Orbit Classification

Problem: Classify satellite orbits from 6 orbital parameters

PCA Approach: 6D $\rightarrow$ 2 PCs for visualization



Result:

- 5 orbit types separate clearly in 2 PCs
- $PC1 \approx$ orbit size (a , period, apogee — Kepler's law)
- $PC2 \approx$ orbit shape & inclination (e, i)

Key Insight:

- 6 parameters are **not independent** — $T \propto a^{3/2}$, apogee/perigee = $a(1 \pm e)$
- PCA discovers these physical couplings automatically
- LEO vs SSO: similar altitude, different inclination — resolved by PC2

Benefit: Catalog classification, debris tracking, conjunction screening

Application 2: Anomaly Detection

Problem: Detect unusual flight conditions from normal operations

Approach:

1. Train PCA on **normal flights only**
2. Project new flight data onto PCs
3. Measure **reconstruction error**
4. High error $\rightarrow$ anomaly!

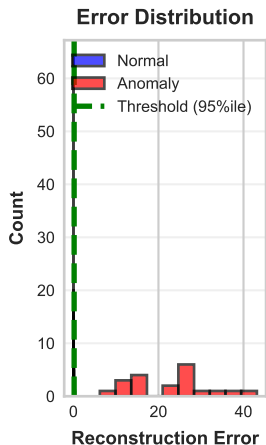
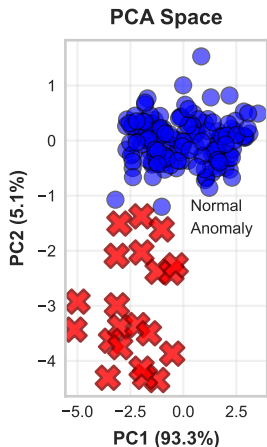
Reconstruction Error:

$$\text{Error} = \|X - \hat{X}\|^2$$

where $\hat{X} = \text{PCA}^{-1}(\text{PCA}(X))$

Intuition: Normal flights reconstruct well, anomalies don't!

Anomaly Detection: Visualization



Left: PCA projection

- Blue: Normal flights (clustered)
- Red X: Anomalies (scattered)

Right: Reconstruction errors

- Normal flights: low error
- Anomalies: high error
- Green line: 95th percentile threshold

Detection Rule:

Error > threshold → Anomaly!

Application: Engine health monitoring, sensor malfunction detection

Application 3: Data Compression

Problem: Store $10,000 \text{ flights} \times 8 \text{ sensors} \times 3600 \text{ time steps} = 288\text{M values}$

PCA Solution (from fleet analysis: 2 PCs capture 95%):

1. Apply PCA: $8\text{D} \rightarrow 2\text{D}$
2. Store 2 PC scores per time step + small loading matrix V_k

Storage Comparison:

Method	Values / Flight	10,000 Flights
Original	$8 \times 3600 = 28,800$	288M values
PCA (k=2)	$2 \times 3600 = 7,200$	72M values
Savings	75%	216M saved

Reconstruct when needed: $\hat{X} = Z_k V_k^T + \bar{X}$

Application 4: Noise Filtering

Insight: Signal lives in high-variance PCs; noise in low-variance PCs!

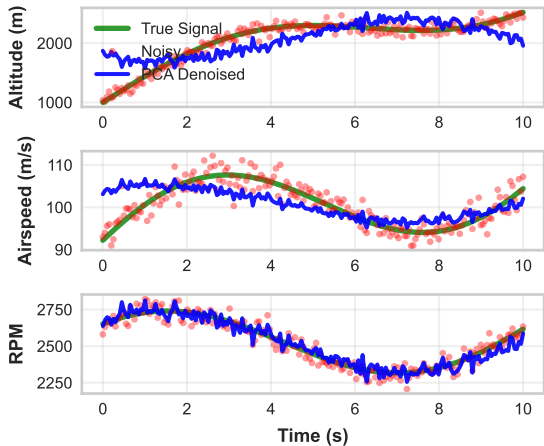
Procedure: PCA $\rightarrow$ keep top k PCs $\rightarrow$ discard the rest $\rightarrow$ reconstruct

$$X_{\text{noisy}} \xrightarrow{\text{PCA}} Z \xrightarrow{\text{Keep top } k} Z_k \xrightarrow{\text{Reconstruct}} X_{\text{denoised}}$$

Application: Clean vibration sensor data, remove electrical noise

Noise Filtering: Example

PCA Noise Filtering (keep PC1 only)



Three Plots (Altitude, Airspeed, RPM):

- **Green:** True signal
- **Red:** Noisy measurements
- **Blue:** PCA-filtered

How It Works: Noise is random (low correlation); signal is structured (high correlation). PCA captures structure, discards noise.

Result: All 3 sensors denoised simultaneously using cross-sensor correlations!

Integration: PCA + Clustering

Why Combine PCA and Clustering?

Problem: Clustering high-dimensional data

Challenges:

1. **Curse of dimensionality:** distances become meaningless
2. **Computational cost:** $O(nkdi)$
3. **No visualization possible**
4. **Correlated features:** redundancy confuses algorithms

PCA Solutions:

1. **Reduce dimensions:** 8D $\rightarrow$ 3D
2. **Remove correlation:** PCs are orthogonal
3. **Enable visualization:** project to 2D/3D
4. **Denoise:** remove low-variance noise

Pipeline: Data $\rightarrow$ Standardize $\rightarrow$ PCA $\rightarrow$ Cluster $\rightarrow$ Interpret

Implementation: PCA + K-Means

```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

# Step 1: Standardize
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

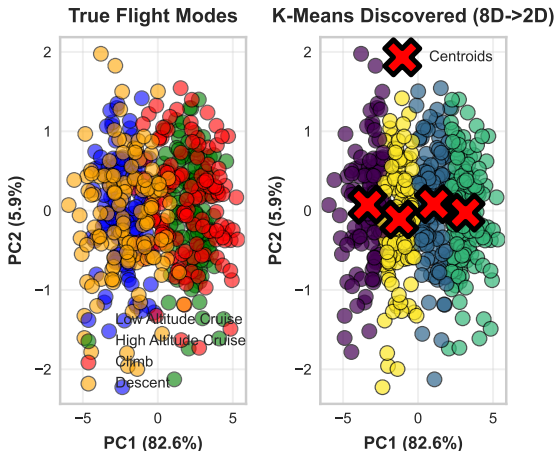
# Step 2: PCA for dimensionality reduction
pca = PCA(n_components=0.95) # Keep 95% variance
X_pca = pca.fit_transform(X_scaled)
print(f"Reduced from {X.shape[1]}D to {X_pca.shape[1]}D")

# Step 3: Clustering on reduced data
kmeans = KMeans(n_clusters=5, random_state=42, n_init=10)
clusters = kmeans.fit_predict(X_pca)

# Step 4: Evaluate
```

Case Study: Flight Mode Discovery

Dataset: 500 flights $\times$ 8 sensors (unlabeled) — discover natural flight modes



Process:

1. **8D data** (unlabeled)
2. **PCA:** 8D $\rightarrow$ 2D (89% variance)
3. **K-means:** $k=4$ clusters
4. **Validate:** Compare with true modes

Results:

- Excellent silhouette + ARI
- Left: True modes; Right: Discovered clusters

Success: Found natural flight modes without labels!

Comparison: Full vs PCA Clustering

Question: Does PCA hurt or help clustering?

```
# Method 1: Cluster in full space (8D)
kmeans_full = KMeans(n_clusters=4, random_state=42)
clusters_full = kmeans_full.fit_predict(X_scaled)
sil_full = silhouette_score(X_scaled, clusters_full)

# Method 2: Cluster in PCA space (2D)
kmeans_pca = KMeans(n_clusters=4, random_state=42)
clusters_pca = kmeans_pca.fit_predict(X_pca)
sil_pca = silhouette_score(X_pca, clusters_pca)

print(f"Full 8D: Silhouette = {sil_full:.3f}")
print(f"PCA 2D: Silhouette = {sil_pca:.3f}")
```

Typical Results:

Method	Dimensions	Silhouette	Speed
Full	8D	0.42	Slower
PCA	2D	0.42	Faster

When to Use PCA Before Clustering

Use PCA + Clustering when:

- ✓ High-dimensional data ($d > 10$)
- ✓ Features are correlated
- ✓ Need visualization
- ✓ Want faster computation
- ✓ Data contains noise

Skip PCA when:

- × Data already low-dimensional ($d \leq 3$)
- × Features are independent
- × Every feature is critical (interpretability)
- × Non-linear relationships (consider kernel PCA)

Best Practice: Try both, compare performance!

Connection to Aerospace Modal Analysis

Structural Dynamics vs PCA:

Concept	Structural Dynamics	PCA
Input	Vibration measurements	Data matrix X
Method	Eigendecomposition of $M^{-1}K$	Eigendecomposition of Σ
Output	Mode shapes ϕ_i	Principal components v_i
Variance	Modal participation	Eigenvalues λ_i
Truncation	Keep dominant modes	Keep high-variance

Summary: PCA Key Concepts

What is PCA?

- New coordinate system aligned with data variance
- Projects data onto principal directions
- Reduces dimensions while preserving information

How to Apply?

1. Always standardize first
2. Fit PCA, examine scree plot
3. Choose k : cumulative variance $\geq 95\%$
4. Interpret loadings

Summary: PCA Applications

Four Applications:

1. **Visualization:** $8D \rightarrow 2D/3D$ plots
2. **Dimensionality Reduction:** Fewer features for ML
3. **Noise Filtering:** Signal in high-variance PCs
4. **Data Compression:** Store fewer numbers

Integration:

- PCA + Classification: reduce features, then classify
- PCA + Clustering: reduce dimensions, then cluster
- PCA + Regression: remove collinearity

Supervised vs Unsupervised

The Complete ML Taxonomy

Method	Type	Input	Output	Example
Regression	Supervised	X, y continuous	$\hat{y} \in \mathbb{R}$	Predict drag coefficient
Classification	Supervised	X, y categorical	$\hat{y} \in \{0, 1, \dots, K-1\}$	Identify fault type
Clustering	Unsupervised	X only	Cluster assignments	Discover patterns
PCA	Unsupervised	X only	Reduced X'	Reduce features

Decision Flowchart

Labeled Data? → Supervised

- Continuous output → Regression
- Categorical output → Classification

No Labels? → Unsupervised

- Find groups → Clustering
- Reduce dimensions → PCA

Hybrid: Use clustering to create labels, then train classifier

When to Use Each

Regression: Continuous predictions

Classification: Categorical decisions

Clustering: Discover patterns

PCA: Reduce dimensions

Key Question: Do you have labels?

- Yes → Supervised (Regression/Classification)
- No → Unsupervised (Clustering/PCA)

Wrap-Up

Key Takeaways

Classification vs Clustering:

- Classification uses labels; Clustering discovers labels

Clustering Algorithms:

- K-means: Fast, requires k , spherical clusters
- DBSCAN: Finds outliers, arbitrary shapes
- Hierarchical: Explores all k , dendrogram
- GMM: Probabilistic, soft assignments

Validation: Elbow method, silhouette score, ARI

Key Takeaways (continued)

PCA:

- Projects to directions of maximum variance
- Connection to modal analysis (eigenvectors!)
- Choose k to preserve 95% variance

Integration:

- PCA before clustering: faster, more stable
- Safety-aware reduction: preserve classification performance
- Visualization: cluster in 3D PC space

Application Context:

- Unsupervised learning discovers unknown failure modes
- PCA reduces feature redundancy
- Always validate physical interpretability

Homework 5 Preview

Assignment: Data Clustering & PCA Analysis

Part 1: Clustering (35 points):

- Implement K-means with elbow method selection
- Apply DBSCAN for anomaly detection
- Hierarchical clustering with dendrogram
- Validate against ground truth labels

Part 2: PCA (35 points):

- Full PCA with scree plot analysis
- Reduce 6D $\rightarrow$ 3D feature space
- Classification-aware validation (preserve performance)
- Interpret principal components physically

Homework 5 Preview (continued)

Part 3: Integration (30 points):

- PCA + Clustering pipeline
- Compare performance: full vs reduced space
- Visualize 3D clustering results
- Technical writeup with domain interpretation

Bonus (10 points):

- Gaussian Mixture Model implementation
- Advanced visualization (interactive 3D plots)
- Novel application

Due: Next week

Dataset: Synthetic multi-sensor data (provided)

Next Week: Week 6

Coming Up: Advanced Dimensionality Reduction

Topics:

- Proper Orthogonal Decomposition (POD)
- Connection between PCA and POD
- t-SNE for visualization
- Autoencoders (neural network approach)
- Applications to CFD data compression

Bridge to Deep Learning: Autoencoders lead to Week 7 (Neural Networks)

Questions & Discussion

Open Floor:

- Clustering vs classification confusion?
- PCA mathematics clarification?
- Applications?
- Homework questions?

Office Hours: See syllabus

Resources:

- Bishop PRML Chapter 9 (Mixture Models)
- Bishop PRML Chapter 12 (PCA)
- Class notes on Canvas

Thank You!

AERO 689: Machine Learning for Aerospace Engineers

Dr. Raktim Bhattacharya
Texas A&M University

Next Lecture: Week 6 - Advanced Dimensionality Reduction